

Flask Framework

Complete Study Notes

Topics Covered:

1. Introduction To Flask Framework
2. Understanding Simple Flask App Skeleton
3. Integrating HTML With Flask Web App
4. Working With HTTP Verbs GET And POST
5. Building Dynamic URL, Variable Rules And Jinja2 Template Engine
6. Working With REST APIs And HTTP Verbs PUT And DELETE

Topic 1: Introduction To Flask Framework

What is Flask?

Flask is a lightweight, micro web framework written in Python. It is designed to make getting started quick and easy, with the ability to scale up to complex applications.

Why Flask?

- Lightweight and minimal — no unnecessary components
- Easy to learn — great for beginners
- Flexible — you choose what libraries to use
- Built-in development server and debugger
- Widely used in REST API development

Installation

```
pip install flask
```

Key Concepts:

- Flask is called a micro-framework because it keeps the core simple
- It is based on Werkzeug (WSGI toolkit) and Jinja2 (template engine)
- Does NOT require a specific database or form library

Topic 2: Simple Flask App Skeleton

Basic App Structure

```
from flask import Flask
app = Flask(__name__)

@app.route('/')
def home():
    return 'Hello, World!'

if __name__ == '__main__':
    app.run(debug=True)
```

Breaking It Down:

- **Flask(__name__)** — creates the Flask app instance; `__name__` tells Flask where to look for resources
- **@app.route('/')** — decorator that maps a URL to a function
- **def home()** — view function that runs when URL is visited
- **return** — every view function MUST return a response
- **app.run(debug=True)** — starts the dev server; `debug=True` shows errors in browser

■ Note: Never use `debug=True` in production!

Running The App:

```
python app.py
```

Then open `http://127.0.0.1:5000` in your browser.

Topic 3: Integrating HTML With Flask

render_template()

Instead of returning plain strings, Flask can return full HTML pages using `render_template()`.

```
from flask import Flask, render_template
app = Flask(__name__)

@app.route('/')
def home():
    return render_template('index.html')
```

Folder Structure (IMPORTANT):

```
your_project/
  app.py
  templates/
    index.html ← HTML files go HERE
```

■ Flask ONLY looks inside the 'templates' folder — spelling must be exact!

Passing Data To HTML:

```
# In Python:
return render_template('index.html', name='Alice', age=25)

# In HTML (index.html):
<h1>Hello, {{ name }}!</h1>
<p>Age: {{ age }}</p>
```

Topic 4: HTTP Verbs GET And POST

GET vs POST

- **GET** — fetches/displays data; parameters visible in URL; used for reading
- **POST** — sends data to server; parameters hidden; used for forms/login

Handling GET and POST in Flask:

```
from flask import Flask, request, render_template

@app.route('/form', methods=['GET', 'POST'])
def form():
    if request.method == 'POST':
        name = request.form.get('name')
        return f'Hello, {name}!'
    return render_template('form.html')
```

HTML Form:

```
<form action='/form' method='POST'>
  <input type='text' name='name'>
  <input type='submit' value='Submit'>
</form>
```

Key Rules:

- Always specify methods=['GET', 'POST'] in the route
- Use request.form.get('fieldname') to read form data
- Every view function MUST return a response — all code paths must have a return
- action='/submit' must start with / (forward slash)

Topic 5: Dynamic URL, Variable Rules & Jinja2

Variable Rules in URL:

```
@app.route('/user/<name>')
def user(name):
    return f'Hello, {name}!'

# With type converter:
@app.route('/score/<int:marks>')
def score(marks):
    return f'Your score: {marks}'
```

Type Converters:

- **<string:name>** — default, accepts any text
- **<int:id>** — accepts integers only
- **<float:price>** — accepts decimal numbers

Jinja2 Template Engine:

- **{{ variable }}** — print/display a variable value
- **{% if %} ... {% endif %}** — conditional statements
- **{% for item in list %} ... {% endfor %}** — loops
- **{# comment #}** — comments (not visible in HTML output)

Jinja2 Example:

```
{% if score >= 50 %}
    <h1>PASSED</h1>
{% else %}
    <h1>FAILED</h1>
{% endif %}

{% for student in students %}
    <p>{{ student }}</p>
{% endfor %}
```

url_for() — Building URLs Dynamically:

```
from flask import url_for, redirect

# Instead of hardcoding '/success':
return redirect(url_for('success', score=85))
```

Topic 6: REST APIs — PUT And DELETE

What is a REST API?

REST (Representational State Transfer) is an architectural style for building APIs. It uses HTTP methods to perform operations on data (resources).

HTTP Methods Summary:

- **GET** — Read / Retrieve data
- **POST** — Create new data
- **PUT** — Update existing data (replaces entire resource)
- **DELETE** — Delete data

Flask REST API Example:

```
from flask import Flask, jsonify, request
app = Flask(__name__)

students = {'1': 'Alice', '2': 'Bob'}

# GET — fetch all
@app.route('/students', methods=['GET'])
def get_students():
    return jsonify(students)

# POST — add new
@app.route('/students', methods=['POST'])
def add_student():
    data = request.get_json()
    students[data['id']] = data['name']
    return jsonify({'message': 'Added'}), 201

# PUT — update
@app.route('/students/<id>', methods=['PUT'])
def update_student(id):
    data = request.get_json()
    students[id] = data['name']
    return jsonify({'message': 'Updated'})

# DELETE — remove
@app.route('/students/<id>', methods=['DELETE'])
def delete_student(id):
    students.pop(id, None)
    return jsonify({'message': 'Deleted'})
```

Key Functions:

- **jsonify()** — converts Python dict to JSON response
- **request.get_json()** — reads incoming JSON data from the request body
- Return status codes: 200 (OK), 201 (Created), 404 (Not Found), 500 (Error)

Testing APIs:

- Use Postman — GUI tool to send GET/POST/PUT/DELETE requests
- Use curl in terminal: `curl -X DELETE http://127.0.0.1:5000/students/1`